Multi-Agent AI Lab: LLM Agents, Retrieval and Resource-Aware Cooperation

How do several agents decide together?

Session 2 · Theory

Julia Schaumeier

IN2106 · WS 2026/27 · 26 October 2026



Today

1. Last week, revisited; your hopes and worries
2. Your specialisations
3. Why several agents?
4. Games between agents
5. Coordination: three protocols
6. How agents are organised
7. Multi-agent LLM systems
8. What M1 asks of you, and beyond

Last week, revisited

Clear

- What is an agent and what is not
- How to classify agents and environments: deterministic or not, for example
- Agents are much more than what the media call agents

Muddy

- What exactly is an agent and which kinds are there?
- How the paradigms differ: too abstract
- How are rewards given?
- How the Archive Games and the specialisations fit together
- How the deliverables and the peer review work

“What is an agent?” is on both lists.

Your hopes and worries

You hoped for		You worried about	
Structures for agents working together, their protocols and trade-offs	today	Too much theory	now
Agents that talk to each other and the architecture	today, M1	No experience; which tools; designing your own agents; explaining every line; relying on AI	at the end
Reliable, consistent results; efficiency in tokens	Session 3	Not knowing why an agent did something	M1
Agents that work in real-world use	throughout	Whether an agent works beyond the data it was built on; evaluating efficiently	Session 3

After M3 we come back to this list: did the course deliver?

By the end of today, you will be able to

1. say when several agents are worth it
2. analyse a simple game between agents
3. name what coordination manages
4. choose a protocol or an organisation for a problem, and justify it
5. recognise how multi-agent LLM systems fail

Your specialisations

4 minutes — in your teams

- Each team: who takes which specialisation?
- Anything still open — say so now



Gate
Player



Night Round
Improviser



Reading Desk
Musicologist



Variorum
Critic

Why several agents?

Multi-agent systems

Definition

"A multiagent system is one that consists of a number of agents, which interact with one-another. In the most general case, agents will be acting on behalf of users with different goals and motivations. To successfully interact, they will require the ability to cooperate, coordinate, and negotiate with each other, much as people do."

- The problem breaks into parts that need different expertise
- The information or the control is naturally distributed
- One part failing does not end the whole

Wooldridge (n.d., Chapter 1 slides, slide 15)

Multi-agent systems in the world



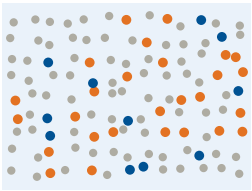
Warehouse robot fleets
Photo: Techwords, CC BY-SA 4.0, via Wikimedia Commons



Drone swarms
Photo: Nylki, public domain, via Wikimedia Commons



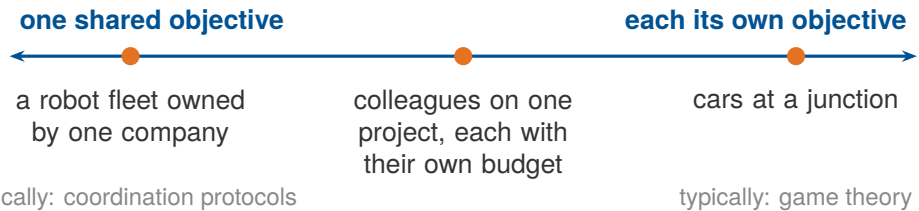
Adaptive traffic signals
Photo: GK tramrunner RU, CC BY-SA 4.0, via Wikimedia Commons



Outbreak simulation
Grey susceptible, orange infected, blue recovered

Most of these have no language model at their core.

Cooperating, competing — or both



Most real systems sit somewhere in between — and need both.

Russell & Norvig (2021, Chapter 17)

Games between agents

A game

- Players, their actions and a payoff for each player for every combination of actions
- A **dominant strategy** is best whatever the others do
- A **Nash equilibrium**: nobody gains by changing only their own action

player 1 \ player 2	cooperate	defect
cooperate	$-1, -1$	$-3, \ 0$
defect	$0, -3$	$-2, -2$

the prisoner's dilemma: years in prison, player 1's first

In the prisoner's dilemma both defect and both do worse than if both had cooperated: individually rational is not the same as collectively good. Played **repeatedly**, cooperation can pay — tit-for-tat famously won a tournament of computer programs.

Shoham & Leyton-Brown (2010, Chapters 3 and 6)

A stranger in Munich

1 minute — on your own

- You have to meet a stranger in Munich tomorrow
- Neither of you can contact the other, and nothing was agreed beforehand
- Write down **where** and **when**

The same, as a game

2 minutes — with a neighbour

Say there are only two places, Marienplatz and the Hauptbahnhof. You and the stranger each score 1 if you meet, 0 if not.

- Write the payoff matrix
- Find the Nash equilibria: the cells where neither of you gains by switching alone
- How do you make sure you end up in an equilibrium?

Coordination: three protocols

Coordination is managing dependencies

Shared resource	two activities draw on one limited thing
Producer–consumer	one activity's output is another's input
Simultaneity	activities that must, or must not, happen at the same time
Task–subtask	a goal that has to be broken down into subtasks

Malone & Crowston (1994)

Coordinate a kitchen

4 minutes — in your teams

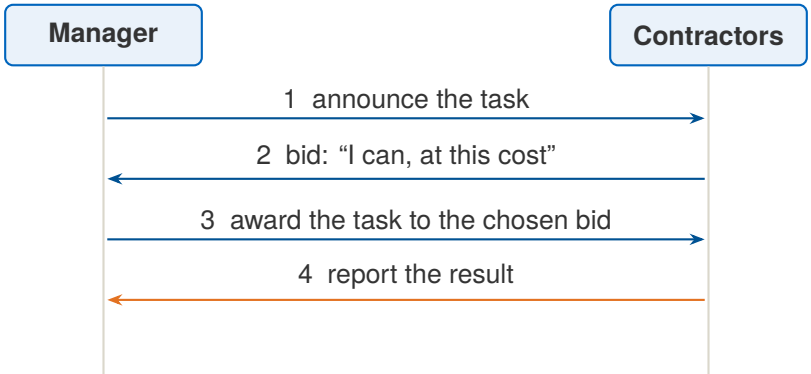
- A restaurant kitchen: a head chef, a grill cook, a pastry cook, a dishwasher, one oven
- Find one example of each kind of dependency: a shared resource, a producer–consumer hand-off, things that must happen at the same time, a task split into subtasks
- The head chef falls ill tonight. What stops working?

Three protocols

Contract Net	Blackboard	Voting
a manager announces a task, agents bid, the manager picks one bid	a shared space agents post to and read from	independent answers, combined by a rule into one decision

A protocol fixes who may say what to whom and when.

Contract Net



Smith (1980)

Contract Net in code

Example: a taxi firm. The dispatcher manages; the taxis bid for rides

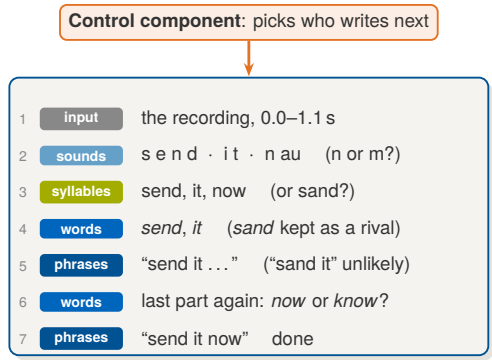
- Only taxis that are free send a bid
- The manager's own rule picks the winner: here the nearest taxi
- Nobody keeps a list of who is free: the bids carry that information
- Fits when many could do the task, capacity varies and tasks keep arriving

```
# Manager: a taxi dispatcher with a new ride
def dispatch(ride):
    broadcast(Announce(ride))                # to every taxi
    bids = collect(Bid, until=deadline)      # offers come in
    if not bids:
        return dispatch_later(ride)          # nobody free
    best = nearest(bids)                     # the rule
    send(best.sender, Award(ride))           # one winner
    return receive(Report, best.sender)      # the result

# Contractor: every taxi runs this loop
while True:
    msg = receive()
    if msg.kind == "announce" and free():
        cost = minutes_to(msg.ride.pickup)
        send(msg.sender, Bid(cost))
    elif msg.kind == "award":
        result = drive(msg.ride)
        send(msg.sender, Report(result))
```

Blackboard

- A shared **board** holds partial results and open guesses
- **Knowledge sources** watch the board and add what they can; they never call each other
- A **control** component decides who writes next
- The solution grows step by step, in no fixed order



The board for a recording of “send it now”, in the style of Hearsay-II (1980), which turned speech into sentences

Erman, Hayes-Roth, Lesser & Reddy (1980)

Blackboard in code

Buys

- sources that need not know about each other
- partial results anyone can build on

Costs

- someone must decide who acts next
- two entries can disagree
- reading the whole board takes time

```
board = Board()                # partial results
board.post(signal)             # the recording

class Words(KnowledgeSource):  # one per level
    def can_help(self, board):   # syllables to join?
        return board.has_unused("syllable")
    def contribute(self, board):
        syl = board.unused("syllable")
        board.post(guess_word(syl))

sources = [Syllables(), Words(), Phrases()]
while not board.has("sentence"):
    ready = able(sources, board)  # those that can help
    if not ready:
        break                  # nobody can build on anything
    chosen = control(ready, board)  # who goes next
    chosen.contribute(board)
```

Voting



- Each voter answers on their own; a rule turns the answers into one decision
- **When the voters want different things**, the rule decides who wins
  - **Plurality**: most first places
  - **Borda count**: points for each place in every ranking
  - **Condorcet winner**: beats every other option head to head, if one exists
  - From the same ballots, these rules can pick different winners
- **When there is a right answer**, voting can find it more often than any one voter
  - Voters need to be better than chance, make independent mistakes $\Rightarrow$ majority is right more often than one voter; stronger results for more voters (Condorcet's jury theorem)
  - Same idea for majority vote in machine learning

Shoham & Leyton-Brown (2010, Chapter 9); List (2022, Section 2.3)

Voting in code

- Each agent answers without seeing the others' answers
- A rule turns the answers into one decision
- Here plurality first, Borda if there is no majority

```
answers = []
for agent in agents:          # each answers on its own
    answers.append(agent.answer(question))

winner, votes = most_frequent(answers)   # plurality
if votes <= len(agents) / 2:             # no majority:
    rankings = rank_all(agents, options) # a stronger rule
    winner = borda(rankings)             # points by place
decide(winner)
```

Nine friends rank where to eat

Friends	4	3	2
1st	Pizza	Curry	Sushi
2nd	Curry	Sushi	Curry
3rd	Sushi	Pizza	Pizza

Same ballots, different winners

Rule	Winner	Why
Plurality	Pizza	most first places, but 5 of 9 rank it last
Borda (2, 1, 0)	Curry	12 points; Pizza 8, Sushi 7
Condorcet	Curry	beats Pizza 5 to 4 and Sushi 7 to 2
The code above	Curry	Pizza has no majority, so Borda

Which protocol?

3 minutes — with a neighbour

1. Three doctors are diagnosing a puzzling case: one reads the scans, one runs the lab tests, one knows the rare diseases
2. You have forecasts for tomorrow's rain from five weather services and must publish one
3. A translation agency works with thirty freelance translators. A new document has just come in

Which of today's three protocols would you use for each and why? Where could it go wrong?

Which protocol? Possible answers

Problem	Protocol	Why	Where it could go wrong
Puzzling case	Blackboard	each builds on the others' findings in the patient record, in an order nobody knows in advance	two findings point to different diseases, and someone has to decide which to follow up
Weather forecast	Voting	five independent candidates, one decision	the services share data and models, so their mistakes are not independent
Translation agency	Contract Net	many translators could do it; who is free and knows the subject changes from job to job, and the bids say so	the cheapest bid comes from someone who does not know the subject; nobody bids when all are busy

Other answers are fine if argued, including that none of the three fits.

How agents are organised

How agents are organised

Organisation: “the collection of roles, relationships, and authority structures which govern [a multi-agent system’s] behavior” — who does what, who decides, who talks to whom

Common shapes:

- **Hierarchy**: decisions flow down, results flow up
- **Team**: agents working together towards a shared goal
- **Market**: allocation by bidding
- **Holonic**: a group of agents that acts, from outside, as one agent — a company to its customers, a robot fleet to the warehouse system

Each shape makes some coordination cheap and other coordination expensive.

Horling & Lesser (2004)

Norms and institutions

- A **norm** says what an agent in a role may, must or must not do
- An **institution**: agents and their norms, with monitoring and sanctions
- **Self-organising**: the agents make, change and enforce the norms themselves

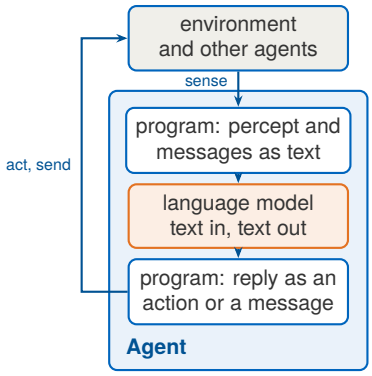
	Farmers sharing an irrigation canal
Permission: may	take water in your turn
Obligation: must	help clean the canal before the season
Prohibition: must not	take water out of turn
Monitoring	the farmer next in line watches the one irrigating
Sanction	a fine, higher for repeat offences
Rules made by	the farmers themselves

Norms coordinate agents nobody designed together; a hierarchy cannot.

Pitt, Schaumeier & Artikis (2012); Ostrom (1990)

Multi-agent LLM systems

A language model as an agent




```
role = "You are the planner. Reply with ..."
history = []
while True:
    percept = sense()          # the environment
    inbox = receive_all()      # other agents' messages
    history += [percept, inbox]
    prompt = role + to_text(history)  # all text
    reply = llm(prompt)        # text in, text out
    action = parse(reply)      # the program reads it
    if action.kind == "message":
        send(action.to, action.text)  # another inbox
    else:
        act(action)
    history.append(reply)
```

The model only ever sees text and only ever writes text. Sensing, acting and talking to others are the program around it.

Multi-agent LLM systems

- Research systems: role-playing pairs (CAMEL); conversation frameworks (AutoGen); simulated societies of agents with memory (Generative Agents)
- Frameworks are mainstream: LangGraph, CrewAI, the OpenAI Agents SDK, Microsoft's Agent Framework (AutoGen's successor)
- Underneath: the same questions as the rest of this session — who decides, what is shared, what it costs
  - A group chat of models $\rightarrow$ a blackboard
  - A planner model hands out subtasks $\rightarrow$ a hierarchy (Contract Net if workers bid)
  - Several models answer, a rule or a judge picks one $\rightarrow$ voting

Guo et al. (2024)

Where multi-agent LLM systems go wrong

- **System design**: agents ignore their task or their role, repeat steps, lose the history, do not know when to stop
- **Misalignment between agents**: input ignored, information held back, the task drifting, no question asked when something is unclear
- **Task verification**: stopping too early; checking the result incompletely, wrongly or not at all

Whether several agents beat a single one depends on the task — and several usually cost more tokens.

Cemri et al. (2025); Anthropic (2025)

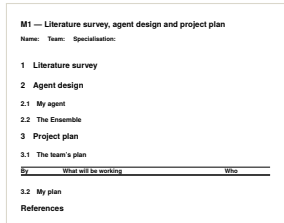
What to take away

- Several agents are worth it when the work, the information or the control is **distributed**
- In a **game**, each agent's rational choice can leave everyone worse off; repetition, conventions and norms help
- **Coordination** is managing dependencies: shared resources, hand-offs, timing, subtasks
- **Contract Net**, a **blackboard** and **voting** each fit some problems and not others
- An **organisation** fixes roles and who decides; its shape sets coordination costs
- Multi-agent LLM systems fail where coordination fails; whether they beat one agent depends on the task

M1

M1 — due Thu 12 Nov, 2 pm

- **Each of you submits one PDF** with your literature survey, your agent's design and your own plan (15%)
- **In it, your team's part**: the Ensemble's design and the team's plan, written together and the same in every member's document (5%)
- **The week after**: your team presents all of the above as one presentation in at most 20 minutes, each of you your own part. Questions and feedback after (10%) — scheduled with each team after the session or by email this week



M1 — Literature survey, agent design and project plan

Name: **Team:** **Specialisation:**

1 Literature survey

2 Agent design

2.1 My agent

2.2 The Ensemble

3 Project plan

3.1 The team's plan

By	What will be working	Who

3.2 My plan

References

The template, without its hints

Each milestone is graded on what it shows at that milestone: whatever you improve later counts at M2 and M3.

How your work fits together

- **Your specialisation is yours**: you design it and build it; at M2 and M3 it plays its own quest alone, graded on your work
- **Your team builds the whole**: one container and a Conductor that coordinates; before M2 you play the Overture together; the system is graded together
- **The full rules** of the four quests and the Overture: Session 4, Mon 9 Nov

My agent: what your design answers

- **The problem**: what can your agent know and what not? What does it spend and what does your quest reward?
- **Your approach and why**: what will you build, compared with at least one alternative from your survey?
- **How you will know it works**: the simplest version first, what you measure, what you change if it falls short
- **Where it meets the team**: what your part needs from the others and what it hands back

No particular architecture is expected of you. A justified choice is what is graded.

The Ensemble: working together

In the Overture, and later the Symphony, the whole Ensemble plays together. As a team, answer:

- **Where** will your parts depend on each other? One dependency of each kind, where you can
- **How** will you coordinate them and why, compared with one alternative?
- **When** one part fails, is slow or wrong, what does the rest do?
- **What** will you log, to tell which part cost what, which went wrong and why?

The handout *The arena at a glance*, on Moodle from today, says what the arena sends and accepts. Session 4 brings the full reference and the client.

The project plan, up to M2

Your team's plan: a table of steps you can check — what will be working, who owns it, by when

- The Conductor's work and who takes which share of it
- When your parts first play the Overture together: before the rehearsal
- How the workload is shared between you

My plan: your own steps, finer than the team's; what is most likely to slip and what you would drop if it does

For later: bring both, updated, to every mentor meeting.

How to hand it in

- One PDF per student, on Moodle, from the template: LaTeX or Word, both there
- At most 3 pages; how you divide them between the parts is up to you
- Figures count towards the limit, references do not
- Your team's two parts are the same in every member's document: we grade them once
- A4, 11 pt, as the template sets it

Beyond M1

Building it: what you may use

- **Libraries that compute are fine**: numerical, machine-learning, graph or solver libraries. Name them like any source
- **Libraries that run your agents for you are not**: the loop, the routing, the memory and the tool calls are your own code
- **The only language model is the course endpoint**
- **Inside your specialisation**, as many agents as you like, built as you like
- **You answer for every line you write**; for a library, know what it does and why you chose it
- **No experience yet?** Start with the simplest version that works. Session 4 brings the code you start from and a self-test

Peer review, across teams

- M2 and M3: all teams present in one joint session. While one team presents, everyone else reviews it
- A short form on Moodle, filled in during the session: what worked well, what could be even better, what you would like to ask them
- The team gets the reviews without names, by email or on Moodle
- Reviewing in good faith earns participation (5%)

One-minute paper

1 minute — on your own

- Orange note: the clearest point today
- Pink note: the muddiest point today
- Leave them on the wall as you go

Further reading

On top of Session 1's list, for today's topics and for your literature survey:

Shoham, Y., & Leyton-Brown, K. (2010). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations* (Rev. 1.1). Cambridge University Press. — free online

Malone, T. W., & Crowston, K. (1994). The interdisciplinary study of coordination. *ACM Computing Surveys*, 26(1), 87–119.

Horling, B., & Lesser, V. (2004). A survey of multi-agent organizational paradigms. *The Knowledge Engineering Review*, 19(4), 281–316.

Cemri, M., et al. (2025). Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems* (NeurIPS 2025, Datasets and Benchmarks Track). arXiv:2503.13657.

Guo, T., et al. (2024). Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of IJCAI-24*, 8048–8057.

Until we meet again

- **This week**: start with your literature survey; as a team, how your Ensemble will work
- **Mon 2 Nov**, Session 3: language models and classical methods
- **Thu 12 Nov, 2 pm**: M1 due

References

References

Anthropic (2025, June 13). How we built our multi-agent research system. *Anthropic Engineering*. https://www.anthropic.com/engineering/multi-agent-research-system

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems* (NeurIPS 2025, Datasets and Benchmarks Track). arXiv:2503.13657.

Erman, L. D., Hayes-Roth, F., Lesser, V. R., & Reddy, D. R. (1980). The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys*, 12(2), 213–253.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24)*, 8048–8057.

Horling, B., & Lesser, V. (2004). A survey of multi-agent organizational paradigms. *The Knowledge Engineering Review*, 19(4), 281–316.

List, C. (2022). Social choice theory. In E. N. Zalta & U. Nodelman (Eds.), *The Stanford Encyclopedia of Philosophy* (Winter 2022 ed.).

References

Malone, T. W., & Crowston, K. (1994). The interdisciplinary study of coordination. *ACM Computing Surveys*, 26(1), 87–119.

Ostrom, E. (1990). *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press.

Pitt, J., Schaumeier, J., & Artikis, A. (2012). Axiomatization of socio-economic principles for self-organizing institutions: Concepts, experiments and challenges. *ACM Transactions on Autonomous and Adaptive Systems*, 7(4), Article 39.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed., global ed.). Pearson.

Shoham, Y., & Leyton-Brown, K. (2010). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations* (Rev. 1.1). Cambridge University Press.

Smith, R. G. (1980). The Contract Net Protocol: High-level communication and control in a distributed problem solver. *IEEE Transactions on Computers*, C-29(12), 1104–1113.

Wooldridge, M. (n.d.). *Chapter 1: Introduction* [Lecture slides]. Slides for *An Introduction to MultiAgent Systems* (2nd ed.). https://www.cs.ox.ac.uk/people/michael.wooldridge/pubs/imas/distrib/pdf-slides/lect01.pdf